

Sentinel-X Yanshee Defender 3000

Rapport d'ingénierie technique du prototype cyber-physique
de surveillance des micro-centrales

Workshop national EPSI Bac+4

Session octobre 2026 · Groupe 7
Workshop2026-M1-G7

Consortium

Infra : Elios, Benoît
Dev : Felis, Martin, Matis

Sommaire

01	Synthèse	2	08	IA de maintenance prédictive	10
02	Architecture et flux	3	09	Infrastructure	12
03	Schéma réseau	4	10	Matrice de sécurité	13
04	Schéma de câblage	5	11	Rapport d'audit	14
05	Firmware ESP8266	6	12	Fabrication et teaser	15
06	API et dashboard	7	13	Organisation	16
07	IA de vision	9	A	Annexe : poster A3	17

La couverture n'est pas numérotée. Le poster A3 (annexe A) est la dernière page du dossier.

01 Synthèse

AetherCorp exploite des micro-centrales isolées, exposées à trois menaces : cyberattaques, intrusions physiques et risques environnementaux (fuite de gaz, surchauffe). Sentinel-X est le boîtier de surveillance autonome qui répond aux trois, relié de façon chiffrée à un centre de commandement.

9 à 22 min

d'avance de l'IA prédictive sur le seuil critique (fuite lente, surchauffe)

0,15 / h

fausses alertes de l'IA prédictive, sur 47 h de fonctionnement normal

< 1 s

entre le clic sur le dashboard et l'alarme du boîtier (télémetrie à 1 Hz)

Ce que fait le prototype

Edge : boîtier ESP8266

Mesure température, humidité, gaz et présence, affiche son état sur un écran OLED, publie chaque seconde en MQTT sur TLS et déclenche buzzer et LED sur commande.

IA de vision

YOLO26n analyse le flux de la caméra du robot Yanshee, compte les personnes, archive une photo de preuve et ordonne la riposte du robot.

IA prédictive

Isolation Forest et Random Forest par capteur. Le modèle repère une dérive lente que le firmware absorbe et en donne le diagnostic, sans aucun seuil codé à la main.

Centre de commandement

Broker Mosquitto, API FastAPI, base MariaDB Galera à 3 nœuds derrière HAProxy, dashboard React temps réel, supervision Zabbix, déploiement Ansible.

Variante d'architecture retenue

Nous avons choisi une **topologie distribuée « Edge-to-Server »**, variante de l'option B du sujet. Le rôle de PC Serveur Local est réparti sur des machines virtuelles Proxmox segmentées en 4 VLAN. Ce choix apporte la haute disponibilité de la base, la supervision et l'automatisation.

La webcam USB est remplacée par la **caméra du robot humanoïde Yanshee**. Le robot sert à la fois de capteur vidéo mobile et d'actionneur de dissuasion. **Cette variante a été validée par les coaches.**

Règle éliminatoire du sujet : interconnexion de tous les blocs. La chaîne complète fonctionne de bout en bout : ESP8266 → Mosquitto (TLS) → API → Galera → dashboard (WebSocket), puis dans l'autre sens dashboard → API → Mosquitto → ESP8266. Les deux IA y sont branchées.

02 Architecture et flux de données

Tous les échanges décisionnels passent par le broker Mosquitto, sur TLS. La vidéo, trop lourde pour MQTT, circule en MJPEG direct.

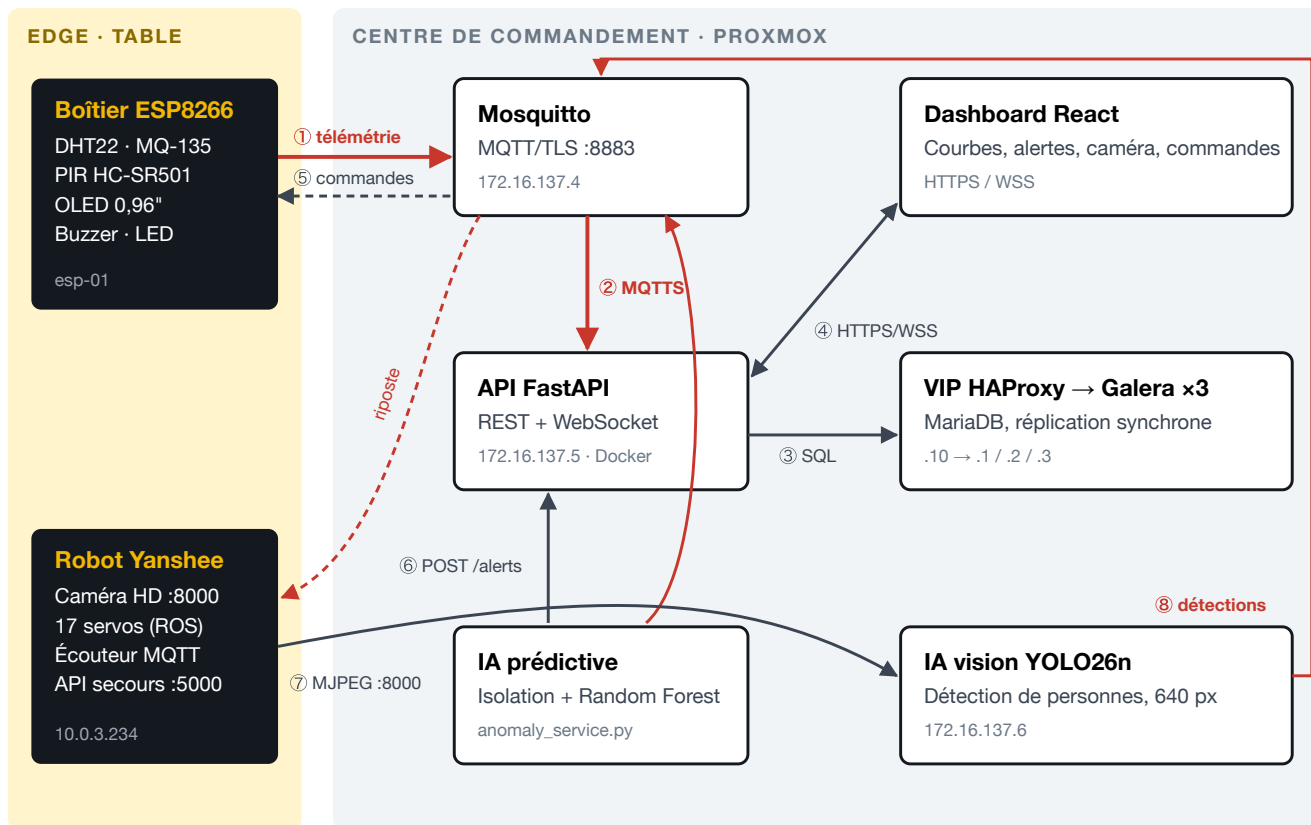


Figure 1. Architecture et flux. En rouge, les flux chiffrés en TLS sur le broker. Les numéros renvoient au tableau ci-dessous.

#	Flux	Protocole	Protection
1	ESP8266 → Mosquitto : mesures 1 Hz, statut (Last Will)	MQTT/TLS 8883	CA embarqué, chaîne vérifiée, authentification
2	Mosquitto → API : ingestion	MQTT/TLS 8883	Authentification broker
3	API → Galera : stockage	SQL 3306 via VIP HAProxy	Utilisateur applicatif, VLAN 20 isolé
4	Dashboard ↔ API : lecture et temps réel	HTTPS + WSS	Reverse proxy TLS
5	Dashboard → API → Mosquitto → ESP8266 : buzzer, LED	HTTPS puis MQTTS	Journal d'audit de chaque commande
6	IA prédictive → API : alertes <code>ia_anomaly</code>	HTTPS POST <code>/api/v1/alerts</code>	Jeton API
7	Robot → IA vision : vidéo 640×480, 25 i/s	MJPEG HTTP 8000	Clé API <code>X-API-KEY</code>
8	IA vision → Mosquitto → API et robot : détections, photo, riposte	MQTT/TLS 8883	CA et nom d'hôte vérifiés

03 Schéma réseau

L'infrastructure est segmentée en quatre VLAN de 62 hôtes. Chaque fonction est isolée, ce qui limite la propagation d'une compromission et rend la matrice des flux explicite.

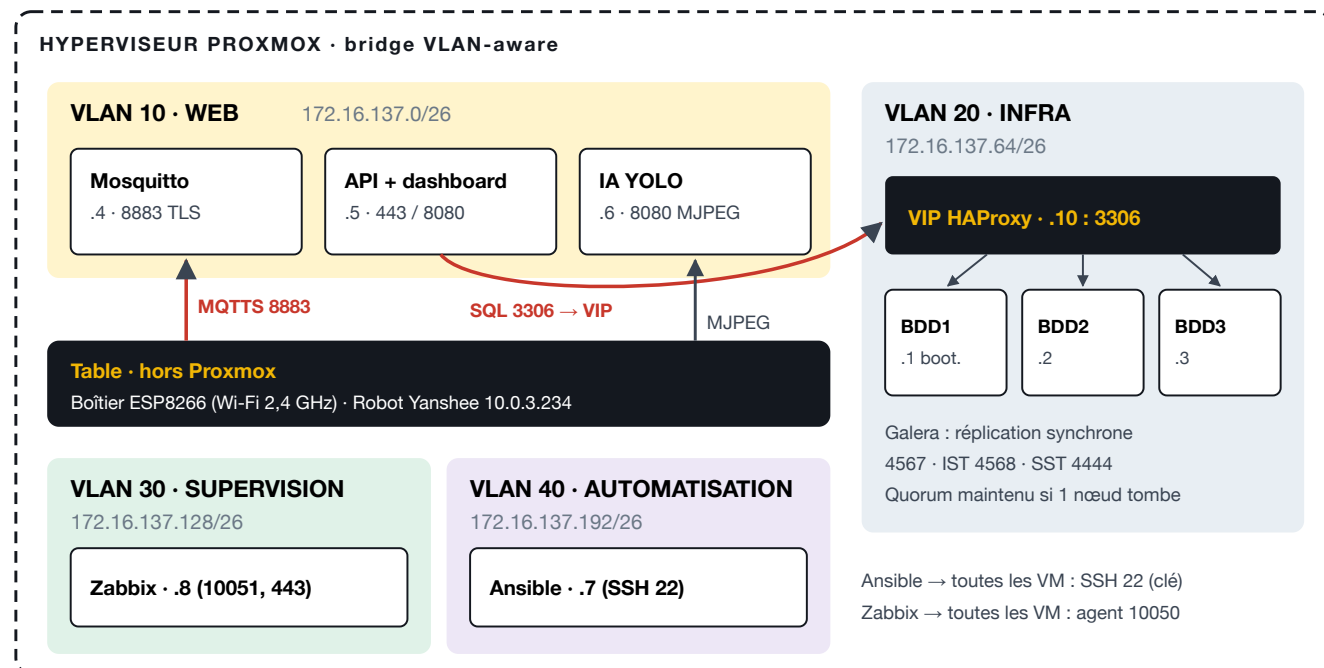


Figure 2. Segmentation réseau. Ports et adresses : `Infra (ports, réseau, IP).xlsx` ; tout port absent de la matrice est fermé par UFW (politique `deny`).

Plan d'adressage

VLAN	Réseau	Passerelle	Machine	IP
10 · WEB	172.16.137.0/26	172.16.137.1	WORKSHOP-GRP7-BDD1 / 2 / 3	172.16.137.1 / .2 / .3
20 · INFRA	172.16.137.64/26	172.16.137.65	WORKSHOP-GRP7-BrockerMosquitto	172.16.137.4
30 · SUPERVISION	172.16.137.128/26	172.16.137.129	WORKSHOP-GRP7-APIDev	172.16.137.5
40 · AUTOMATISATION	172.16.137.192/26	172.16.137.193	WORKSHOP-GRP7-YOLO	172.16.137.6
			WORKSHOP-GRP7-Ansible	172.16.137.7
			WORKSHOP-GRP7-Zabbix	172.16.137.8
			VIP HAProxy	172.16.137.10

Matrice des flux et ports ouverts

Machine	Ports entrants	Usage
Mosquitto	22, 8883, 9001, 80/443, 10050	SSH, MQTTS, MQTT WebSocket, supervision
API + dashboard	22, 443, 8080, 10050	SSH, dashboard HTTPS, API REST + WebSocket
IA YOLO	22, 8080, 10050	SSH, flux annoté MJPEG
Galera (x3)	22, 3306, 4567, 4568, 4444, 10050	SSH, MariaDB, réplication Galera
Zabbix	22, 443, 10051	SSH, interface web, trapper
Ansible	22	SSH

04 Schéma de câblage électronique

Carte NodeMCU ESP8266 (clone Lolin V3, puce USB CH340). Les broches de démarrage D3, D4 et D8 sont laissées libres : un module qui les tire au mauvais niveau empêche le flashage. C'est pour cela que l'OLED est sur D6/D7 en I2C logiciel.

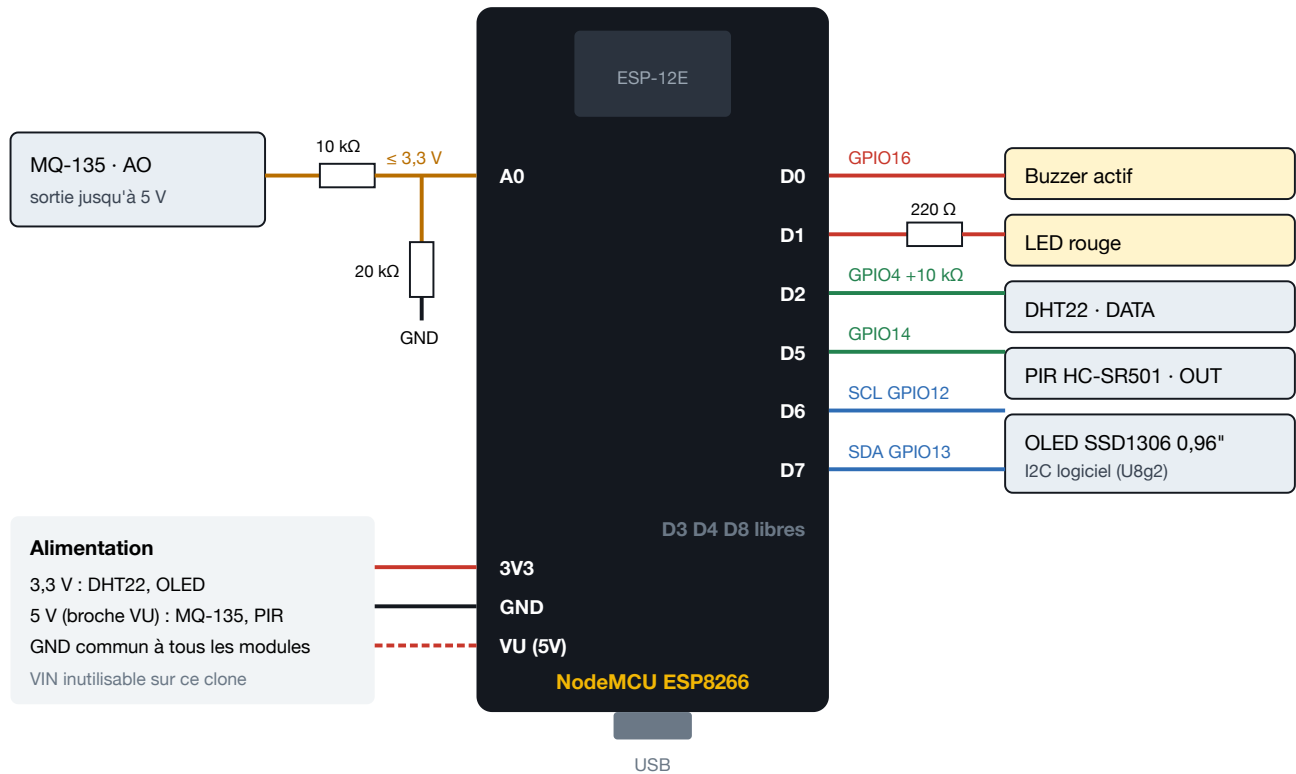


Figure 3. Câblage du boîtier. Le pont diviseur 10 kΩ / 20 kΩ ramène la sortie 5 V du MQ-135 sous les 3,3 V admissibles par l'entrée A0.

Élément	Broche	GPIO	Alim.	Remarque
Buzzer actif	D0	16	—	HIGH = son
LED rouge	D1	5	—	Résistance 220 Ω en série
DHT22 (DATA)	D2	4	3,3 V	Pull-up 10 kΩ entre VCC et DATA
PIR HC-SR501 (OUT)	D5	14	5 V	Sortie 3,3 V, branchement direct
OLED SCL / SDA	D6 / D7	12 / 13	3,3 V	I2C logiciel (U8g2)
MQ-135 (AO)	A0	ADC	5 V	Pont diviseur 10 kΩ / 20 kΩ

Le sujet mentionne un MQ-2. Le boîtier utilise un MQ-135 (gaz et fumées), même brochage analogique et même logique de détection relative.

05 Firmware ESP8266

C++ sous PlatformIO (Arduino), environ 450 lignes. La boucle principale ne contient aucun `delay()` : chaque tâche a sa propre cadence. Les capteurs tournent donc même sans réseau, et les commandes reçues sont appliquées immédiatement.

Tâche	Cadence
Lecture DHT22 (valeurs hors plage rejetées)	2 s
Échantillon gaz, moyenne glissante	100 ms
Niveau gaz (référence adaptative)	1 s
PIR (après 30 s de calibration)	continu
Publication MQTT	1 s
Rafraîchissement OLED	500 ms
Reconnexion broker	5 s

Écran OLED

IP de la carte, température et humidité, niveau de gaz (ou compte à rebours de chauffe), présence, état de la connexion MQTT.

Empreinte

RAM 39,2 % (32 ko sur 80 ko) · Flash 40,1 % (419 ko sur 1 Mo).
CPU à 160 MHz pour accélérer la poignée de main TLS.

Connexion sécurisée au broker

- **MQTT sur TLS uniquement** (port 8883), via BearSSL. Le certificat de l'autorité qui a signé celui du broker est embarqué dans `mqtt_ca.h`, et la chaîne est vérifiée à chaque connexion.
- BearSSL a besoin de l'heure pour valider un certificat. Le firmware la prend par NTP, et à défaut (réseau sans Internet) utilise une date de secours postérieure à la création du CA.
- Authentification par utilisateur et mot de passe. Les identifiants sont dans `secrets.h`, exclu du dépôt ; un modèle `secrets.example.h` est fourni.
- **Last Will** : le broker publie `offline` sur `sentinelx/esp-01/status` si le boîtier disparaît. Le tableau de bord détecte donc une coupure sans attendre.
- Reconnexion automatique du Wi-Fi et du broker.

Détection de gaz

Le capteur ne donne pas une concentration absolue. Le firmware mesure donc l'**écart à l'air calme** : après 60 s de chauffe, il mémorise une référence qui suit ensuite la dérive lente du capteur, mais reste figée en alerte pour ne pas absorber une vraie fuite.

Niveau	Condition	Sortie de niveau
normal	écart $\leq$ 150	—
e leve	écart $>$ 150	redescendre sous 120 (hystérésis de 30)
alerte	écart $>$ 300	redescendre sous 270

Ces seuils sont un premier rempart local, rapide sur une fuite franche. Leur limite (une dérive lente est absorbée par la référence) est justement ce que traite l'IA prédictive (section 08).

Payloads

```
// Télémétrie, toutes les secondes : sentinelx/esp-01/telemetry
{"device":"esp-01","uptime_s":742,"temperature":24.6,"humidity":48.2,
 "gas":{"raw":312,"baseline":290,"level":"normal"},"presence":false,"rssi":-61,
 "actuators":{"buzzer":"off","led":"off"}}

// Commande : sentinelx/esp-01/cmd
{"target":"all","state":"on","duration_ms":3000}
```

En mode alerte, le firmware fait clignoter buzzer et LED en phase (0,5 s allumé, 0,5 s éteint). Avec `duration_ms`, l'alarme s'arrête seule. Le boîtier ne déclenche **aucune alarme de lui-même** : la décision appartient au centre de commandement.

06 API et dashboard

L'API (Python, FastAPI, asynchrone) centralise les métriques et l'historique des événements. Elle ingère MQTT, écrit dans Galera et pousse chaque événement au dashboard par WebSocket.

Routes principales

Méthode	Route	Rôle
POST	/api/v1/alerts	Imposé par le sujet. Réception des alertes capteurs et IA, validation stricte du schéma (201, 422)
GET	/api/v1/alerts	Alertes récentes, filtres par niveau, source, acquittement
PUT	/api/v1/alerts/{id}/acknowledge	Acquittement par le superviseur (jeton requis)
GET	/api/v1/measurements	Historique pour les courbes
GET	/api/v1/status	État du broker, de la base, des boîtiers et de l'IA
POST	/api/v1/commands	Commande buzzer, LED ou robot, relayée en MQTT (QoS 1) et journalisée
GET	/api/v1/commands	Journal d'audit des commandes
POST	/api/v1/auth/login	Obtention du jeton
GET	/api/v1/camera/stream, /photos	Flux vidéo relayé, photos d'intrusion stockées en base
WS	/ws	Événements temps réel : <code>telemetry</code> , <code>alert</code> , <code>anomaly</code> , <code>person_count</code> , <code>photo</code> , <code>robot_action</code>

Contrat d'interface MQTT

Topic	Sens	Contenu
sentinelx/esp-01/telemetry	ESP → serveur	Mesures JSON, 1 par seconde
sentinelx/esp-01/status	ESP → serveur	<code>online</code> / <code>offline</code> (retained, Last Will)
sentinelx/esp-01/cmd	serveur → ESP	<code>{"target":"buzzer led all","state":"on off toggle","duration_ms":3000}</code>
sentinelx/esp-01/anomaly	IA → API	Indice de risque par capteur, diagnostic
detection_robot/person_count, /photo, /timestamp	YOLO → API	Nombre de personnes, JPEG de preuve, horodatage
detection_robot/command	YOLO / API → robot	<code>{"cmd":"punch","dry_run":false}</code>
detection_robot/action_status	robot → API	<code>executing</code> / <code>completed</code>

Modèle de données

Table	Contenu
telemetry	Mesure horodatée : température, humidité, gaz (brut, référence, niveau), présence, RSSI, état des actionneurs
alerts	Source (<code>sensor</code> , <code>ia_vision</code> , <code>ia_anomaly</code>), type, niveau, message, payload, acquittement
actuator_audit	Qui a déclenché quoi, quand, avec quel payload
captured_photos	Photo d'intrusion (base64), nombre de personnes

Déploiement

Image Docker `python:3.11-slim`, Docker Compose avec `restart: unless-stopped` et healthcheck sur `/health`. Les secrets sont lus depuis un `.env` hors dépôt ; Compose refuse de démarrer s'ils manquent. 13 tests automatisés (pytest) couvrent alertes, actionneurs et télémétrie.

Dashboard de supervision

React 19, Vite et Recharts. Le dashboard charge l'historique au démarrage puis se met à jour uniquement par WebSocket, ce qui le rend réactif sans interroger l'API en boucle.

Capteurs

Courbes température, humidité, gaz et RSSI ; présence ; statut du boîtier.

Alertes

Bandeau rouge sur alerte active, liste repliable des dernières alertes.

IA

Flux caméra, compteur de personnes, galerie de photos, actions du robot, jauge de risque prédictif.

Panneau de contrôle

Buzzer (3 s), LED (3 s), alerte totale buzzer + LED, arrêt d'urgence. Chaque clic produit un `POST /api/v1/commands`, une publication MQTT vers `sentinelx/esp-01/cmd` et une ligne d'audit.

Capture d'écran du dashboard en fonctionnement : à insérer ([sources/img/dashboard.png](#))

07 IA de vision

Script Python autonome (`yanshi_video.py`) sur la VM IA. Il lit le flux MJPEG de la caméra du robot, détecte les personnes avec YOLO et publie les événements sur le broker.

Paramètre	Valeur	Pourquoi
Modèle	YOLO26n (Ultralytics), variante « nano »	Le plus léger de la famille, adapté à une inférence CPU temps réel
Classe	COCO 0 (person) uniquement	Seule la présence humaine compte, le reste est filtré à l'inférence
Taille d'entrée	640 px (<code>imgsz=640</code>)	Redimensionnement en amont, exigé par le sujet
Source	640×480, 25 i/s, encodé par le GPU du robot	Aucune charge processeur côté robot
Seuil de confiance	0,40	Compromis entre fausses détections et intrus manqués
Photo de preuve	au plus 1 toutes les 10 s	Ne pas saturer le broker
Riposte du robot	au plus 1 toutes les 5 s	Anti-rebond

Chaîne de traitement

```
Caméra Yanshee -MJPEG-> YOLO26n -> MQTTS person_count -> API -> alerte d'intrusion
                                |
                                +-> MQTTS photo (JPEG) -> API -> Galera + galerie
                                |
                                +-> MQTTS command -> robot (riposte)
                                |
                                +-> HTTP :8080 /stream.mjpg (flux annoté, clé API)
```

- Connexion au broker en TLS avec vérification du CA **et** du nom d'hôte. Le script refuse de démarrer si le CA est absent, plutôt que de se connecter sans vérification.
- L'API transforme un nombre de personnes non nul en alerte d'intrusion, avec un anti-rebond de 5 s.
- Le robot exécute la riposte (bras levés, direct du gauche, direct du droit) et publie son avancement sur `detection_robot/action_status`. Le mode `dry_run` permet de tester sans solliciter les servos.

Latence d'inférence. L'objectif du sujet est moins de 100 ms par trame. Mesure à relever sur la VM IA pendant la démo (`result.speed["inference"]` d'Ultralytics) : ____ **ms** en moyenne.

08 IA de maintenance prédictive

Le sujet interdit les simples conditions du type `if temp > 40`. Notre module apprend le comportement normal de chaque capteur et détecte une dérive **avant** qu'elle n'atteigne un seuil critique. Aucune décision ne repose sur une valeur de capteur codée à la main.

Pourquoi le firmware ne suffit pas

En rejouant exactement la logique gaz du firmware en Python (`firmware_gas.py`), nous avons constaté deux angles morts :

- **Fuite lente** : la référence adaptative rattrape 5 % de l'écart par seconde. Une fuite de +12/min laisse un écart stable d'environ 4 : le firmware reste `normal` pendant toute la fuite.
- **Fuite installée** : au-dessus de 150, la référence remonte lentement. Une fuite est absorbée en 5 min environ, puis le firmware repasse `normal` alors qu'elle continue.

Le firmware n'a par ailleurs aucune alerte de température.

Modèle

Isolation Forest (non supervisé)

Un modèle par capteur, entraîné uniquement sur du fonctionnement normal. Il signale tout comportement inhabituel, y compris un incident jamais vu.

Random Forest (supervisé)

Entraîné sur des incidents simulés étiquetés. Il donne le diagnostic : surchauffe, fuite lente, fuite franche, humidité. Il reste formel quand l'incident est installé.

Features, calculées chaque seconde sur un tampon glissant : valeur, écart à la médiane sur 5 min, vitesse de variation, écart-type sur 10 s, et un écart à une référence long terme qui redescend vite mais ne remonte qu'en une heure. Cette mémoire longue permet de voir un incident que le firmware a absorbé.

Décision : indice de risque normalisé (1 = seuil appris sur des données normales de validation, calé pour 0,05 % de secondes faussement anormales). Alerte après 8 s consécutives au-dessus de 1, diagnostic confirmé après 8 s stables, retour à la normale après 20 s sous 1.

Résultats

Évalués sur des données inédites pour le modèle, avec exactement la logique du service temps réel. 20 tirages par scénario, valeurs médianes.

Scénario	Détection IA	Diagnostic correct	Firmware	Avance sur le seuil critique
Surchauffe lente (+0,3 à 0,8 °C/min)	71 s	20/20	jamais	22 min
Fuite de gaz lente (+5 à 20 /min)	61 s	20/20	jamais	9 min
Fuite de gaz franche	7 s (critical)	20/20	4 s	— (firmware 3 s plus rapide)
Hausse d'humidité (+6 à 35 %)	15 s	19/20	jamais	—
Démo : chauffe lente + micro-dérive de gaz	99 s	20/20	jamais	18 min

Avance médiane de l'IA sur le seuil critique (minutes)

Surchauffe lente	22
Démo chauffe + gaz	18
Fuite de gaz lente	9
Firmware seul	ne détecte jamais ces dérives lentes

0,15 / h

fausses alertes sur 47 h de normal simulé

1,2 %

risque d'une fausse alerte pendant une démo de 5 min

10 / 10

tests automatisés, dont l'équivalence direct / entraînement

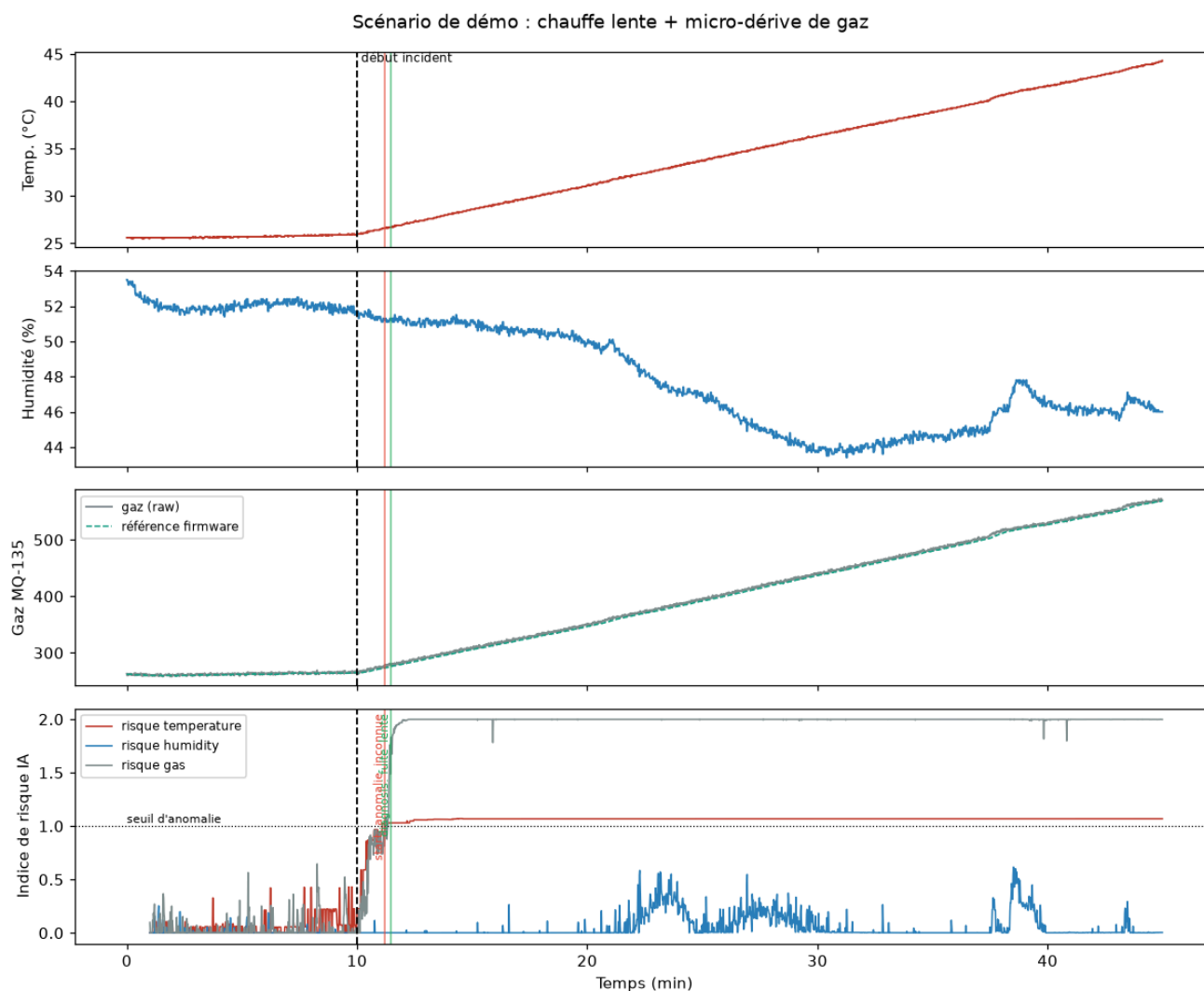


Figure 4. Scénario de démonstration. L'incident commence à 10 min. Le gaz monte lentement et la référence du firmware (pointillés) le suit : le firmware reste à « normal ». L'indice de risque IA (bas) franchit le seuil vers 11 min et pose le diagnostic de fuite lente.

Données et validation

- Le fonctionnement normal est simulé, calibré sur le bruit réel des capteurs (24 h de normal, 78 h d'incidents). Un enregistrement réel de 10 min du 06/10, jamais vu à l'entraînement, est correctement interprété : surchauffe pendant la chauffe à 31,6 °C, humidité au souffle, fuite franche au pic de gaz, puis retour à la normale.
- Sur une fuite brutale, l'anti-rebond coûte 3 s face au firmware. C'est voulu : le firmware reste le premier rempart, l'IA apporte le diagnostic et le niveau critique.
- Le modèle se réentraîne en une minute sur des données de la salle de démo (`data_take.py` puis `train.py`).

Intégration

`anomaly_service.py` s'abonne à `sentinelx/+/telemetry`. Il publie chaque seconde l'indice de risque sur `sentinelx/<device>/anomaly` (affiché en jauge sur le dashboard) et envoie les alertes en `POST /api/v1/alerts`, avec la source `ia_anomaly`. Une simulation publiée sur le broker sert de démo de secours si le boîtier tombe en panne.

Toute la pile serveur tourne sur l'hyperviseur Proxmox de l'école, en conteneurs LXC et machines virtuelles, déployée et supervisée de façon centralisée.

Base de données : MariaDB Galera

3 nœuds (bootstrap + 2 réplicas), chacun en LXC 2 vCPU, 2 Go RAM, 50 Go. Réplication synchrone. Avec 3 nœuds, le quorum est maintenu si l'un tombe, ce qui n'est pas le cas à 2. L'API n'adresse jamais un nœud directement : elle passe par la **VIP HAProxy** (172.16.137.10), qui répartit vers les nœuds disponibles.

Broker : Eclipse Mosquitto

Conteneur Docker. Phase 1 : validation en clair sur 1883. Phase 2 : TLS sur 8883 avec un certificat signé par notre autorité interne, dont le CA est embarqué dans l'ESP8266 et les clients Python. Authentification activée.

Applicatif

VM API (FastAPI en Docker Compose, dashboard derrière un reverse proxy HTTPS) et VM IA (YOLO en service systemd). Accès réservé aux clés SSH personnelles de chaque membre.

Automatisation : Ansible

Serveur dédié. Rôles `bdd`, `galerabootstrap`, `replica1`, `replica2`, `zabbix`, `zabbixagent`. Secrets dans `vault.yml`, à chiffrer avec `ansible-vault`, jamais versionnés.

Supervision et MCO : Zabbix

Zabbix 7.0 (PostgreSQL, nginx), déployé par Ansible ainsi que les agents sur toutes les machines : les 3 nœuds Galera, le broker, l'API, la VM IA et Ansible. Il suit le CPU, la mémoire, le disque et la disponibilité, ce qui répond à l'exigence de maintien en condition opérationnelle face à l'afflux continu de messages MQTT.

Capture du tableau de bord Zabbix : à insérer (<sources/img/zabbix.png>)

Évolutions prévues

- Sauvegardes automatisées de la base.
- Passage à un cluster Mosquitto pour supprimer ce point de défaillance unique.
- Alertes Zabbix par e-mail ou messagerie, et suivi du volume des journaux MQTT avec rotation.
- Rôles Ansible pour Mosquitto, l'API et le durcissement, pour reconstruire toute la pile en une commande.

10 Matrice de sécurité

Mesures de sécurité en place, et comment les prouver devant le jury.

Chiffrement (TLS)

Flux	Mesure	Preuve	État
ESP8266 → Mosquitto	MQTT exclusivement sur TLS 8883, CA embarqué, chaîne du broker vérifiée	Wireshark : trafic illisible	En place
IA vision → Mosquitto	TLS, vérification du CA et du nom d'hôte	<code>MQTT_TLS_INSECURE=false</code>	En place
API et robot → Mosquitto	MQTT sur TLS 8883	Wireshark	En place
Dashboard ↔ API	HTTPS et WSS derrière reverse proxy	Cadenas du navigateur	En place

Durcissement système (hardening)

Exigence	Mesure	Preuve	État
Fermeture des ports non requis	UFW, politique <code>deny</code> par défaut, ports limités à la matrice §03	<code>sudo ufw status verbose</code>	En place
SSH sans mot de passe	Clés asymétriques personnelles, <code>PasswordAuthentication no</code>	<code>sudo sshd -T grep passwordauth</code>	En place
Segmentation réseau	4 VLAN, base de données isolée derrière une VIP	Schéma §03	En place
Isolation des services	Services en conteneurs Docker sans <code>--privileged</code> , une VM par fonction	<code>docker inspect</code>	En place
Haute disponibilité	Galera 3 nœuds, HAProxy, <code>restart: unless-stopped</code> , <code>healthcheck</code>	Arrêt d'un nœud en direct	En place
Supervision (MCO)	Zabbix sur toutes les machines : CPU, RAM, disque, disponibilité	Tableau de bord Zabbix	En place

Authentification et secrets

Élément	Mesure	Preuve	État
Broker MQTT	Authentification obligatoire, aucune connexion anonyme	<code>mosquitto.conf</code>	En place
API : actions du superviseur	Acquittement des alertes protégé par jeton Bearer ou <code>X-API-Key</code> , délivré par <code>/api/v1/auth/login</code>	401 sans jeton	En place
Flux vidéo et API du robot	Clé API obligatoire ; les services refusent de démarrer sans clé	401 sans clé	En place
Secrets	Aucun secret dans le dépôt : <code>.env</code> , <code>secrets.h</code> et <code>vault.yml</code> hors dépôt, modèles <code>*.example</code>	<code>git grep</code> sans résultat	En place
Traçabilité	Chaque commande d'actionneur journalisée (qui, quoi, quand)	<code>GET /api/v1/commands</code>	En place

11 Rapport d'audit de sécurité

Le rapport se compose de deux parties : un auto-audit du code et de la configuration, mené le 8 octobre avant le pentest croisé, et les résultats du pentest croisé de l'après-midi.

11.1 Méthode de l'auto-audit

- Revue de code de toutes les briques : firmware, API, dashboard, IA, scripts du robot, rôles Ansible.
- Recherche de secrets dans les fichiers versionnés (`git grep`).
- Revue de la matrice des ports et des configurations TLS.
- Exécution des suites de tests : API 13/13, IA prédictive 10/10, compilation du firmware.

11.2 Correctifs appliqués avant le pentest

ID	Constat	Correctif	État
A-01	Le boîtier pouvait encore se connecter au broker en clair (1883).	Firmware en MQTT exclusivement sur TLS 8883, avec CA embarqué et vérification de la chaîne.	Corrigé
A-02	Identifiants présents dans des fichiers versionnés.	Secrets sortis du dépôt (<code>.env</code> , <code>secrets.h</code> , <code>vault.yml</code>), modèles <code>*.example</code> , <code>.gitignore</code> étendu.	Corrigé
A-03	Des services du robot démarraient avec une clé API par défaut.	Clé obligatoire : sans <code>SENTINEL_API_KEY</code> , les services refusent de démarrer.	Corrigé
A-04	Docker Compose démarrait l'API avec des valeurs de secours.	Compose exige désormais chaque secret et refuse de démarrer s'il manque.	Corrigé
A-05	Le dashboard affichait le flux brut du robot.	L'API relaie le flux annoté par YOLO, avec le flux du robot en repli.	Corrigé

11.3 Pentest croisé du 8 octobre

Conduit l'après-midi avec Nmap, Wireshark et Metasploit. Attaques visées par le sujet : homme du milieu, injection de payloads, déni de service.

Attaques subies

Heure	Groupe	Outil / technique	Cible	Résultat	Détectée ?

Attaques menées

Heure	Groupe cible	Outil / technique	Cible	Résultat	Succès ?

Bilan et correctifs post-pentest

ID	Vulnérabilité exploitée	Correctif appliqué	État

12 Fabrication et teaser

Boîtier imprimé en 3D

Coque modélisée sous Fusion360, imprimée sur Creality K2 Plus. Cahier des charges : encapsuler l'ESP8266 et les capteurs, laisser apparaître l'écran OLED, ménager des passe-câbles propres sans fil apparent, et ventiler le capteur de gaz (qui chauffe) et le DHT22 pour garder des mesures fidèles.

Rendu Fusion360 : à insérer

Photo du boîtier imprimé : à insérer

Gravure laser

Creality Falcon A1 : logo AetherCorp, consignes de sécurité et numéro de série du module, pour donner au boîtier l'aspect d'un produit fini.

Photo de la gravure : à insérer

Teaser « Sentinel Drop »

Temps	Séquence
0 – 10 s	Hook métier : la menace sur les centrales, ambiance d'alerte
10 – 30 s	Le produit : gros plans du boîtier imprimé et gravé, OLED et capteurs
30 – 50 s	La pile technique en incrustation fond vert : schémas, code, vision IA isolant un intrus
50 – 60 s	Outro : « Sentinel-X : la sécurité à la bordure. »

Format livré : MP4 vertical 1080×1920 (9:16), H.264, 60 s maximum.

13 Organisation

Pôle Infra : Elios, Benoit

Proxmox, VLAN et adressage, cluster Galera et HAProxy, broker Mosquitto et TLS, Ansible, Zabbix, durcissement des serveurs.

Pôle Dev : Felis, Martin, Matis

Firmware ESP8266, API REST et WebSocket, dashboard React, IA de vision et IA prédictive, intégration du robot Yanshee.

Méthode

- Contrat d'interface figé le lundi : topics MQTT, payloads JSON, routes de l'API.
- Suivi des tâches dans Jira, échanges sur Discord.
- Git : une branche par brique, fusion par pull request, commits sémantiques (`feat(...)` , `fix(...)`).
- Tests automatisés sur l'API, l'IA prédictive et la vision.

Planning réalisé

Jour	Réalisations
Lundi 5	Constitution de l'équipe, architecture et flux, contrat d'interface, plan d'adressage, VM et accès SSH
Mardi 6	Câblage, firmware, Mosquitto, Galera par Ansible, API, premières mesures réelles, entraînement IA
Mercredi 7	Intégration de bout en bout, dashboard temps réel, IA sur le flux vidéo, commandes à distance, robot, tournage du teaser
Jeudi 8	MQTT exclusivement sur TLS, retrait des secrets, auto-audit, pentest croisé, livrables

SENTINEL-X

Yanshee Defender 3000 · la sécurité à la bordure.

Workshop EPSI Bac+4 · Groupe 7
Elios · Benoît · Felis · Martin · Matis

01 Trois menaces, une sentinelle

INTRUSION PHYSIQUE
Espionnage industriel

Vision IA + robot
YOLO26n repère toute présence humaine dans le flux du robot Yanshee, archive une photo de preuve et déclenche sa riposte.

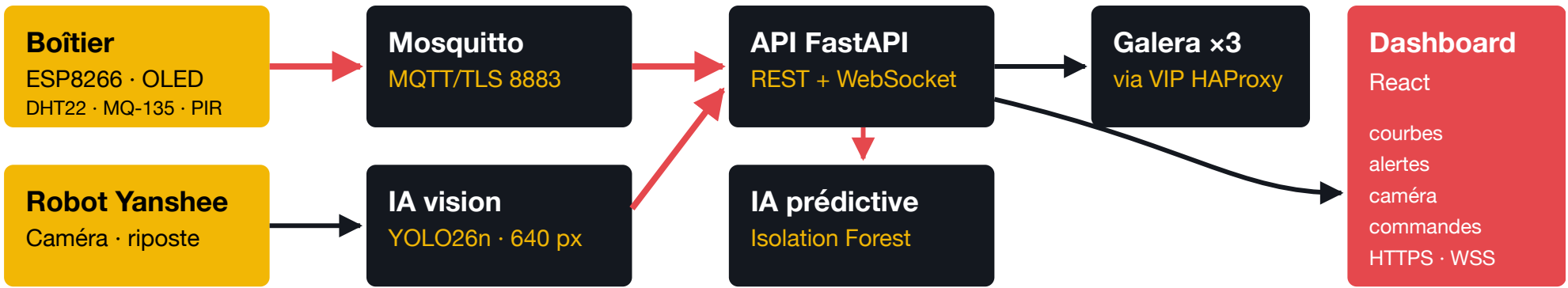
RISQUE ENVIRONNEMENTAL
Fuite de gaz, surchauffe

Capteurs + IA prédictive
Le boîtier mesure température, humidité, gaz et présence. L'IA détecte la dérive lente **avant** le seuil critique.

CYBERATTAQUE
Déstabilisation réseau

Chiffrement + durcissement
MQTT sur TLS avec CA embarqué, HTTPS, pare-feu UFW, SSH par clé, réseau segmenté en 4 VLAN.

02 De la bordure au centre de commandement



22 min
d'avance de l'IA sur le seuil critique d'une surchauffe lente

0,15/h
fausses alertes de l'IA prédictive (47 h de test)

< 1 s
du clic sur le dashboard à l'alarme du boîtier

3 nœuds
Galera : la base survit à la perte d'un serveur

L'IA voit ce que le seuil ne voit pas

Pas de if temp > 40 : un Isolation Forest et un Random Forest par capteur apprennent le normal et diagnostiquent la dérive.

Surchauffe lente		22 min
Chauffe + dérive gaz		18 min
Fuite de gaz lente		9 min
Firmware seul	ne voit jamais ces dérives	

Médiane sur 20 scénarios inédits pour le modèle.

La pile

C++ / PlatformIOMosquittoFastAPIReactYOLO26nscikit-learnMariaDB GaleraHAProxyDockerProxmoxAnsibleZabbixROS Fusion360

Sécurité de bout en bout

- MQTT exclusivement sur TLS 8883, CA embarqué dans l'ESP8266
- Dashboard et API en HTTPS / WSS
- Pare-feu UFW : tout fermé sauf la matrice des flux
- SSH par clé uniquement, aucun mot de passe
- 4 VLAN : web, base de données, supervision, automatisation
- Supervision Zabbix de toutes les machines
- Déploiement Ansible, secrets hors dépôt
- Auto-audit et pentest croisé documentés